

Data Engineer SQL Interview Guide

A comprehensive collection of SQL questions from basic to advanced, with detailed explanations from a data engineering perspective.

Table of Contents

- [Easy Level Questions](#)
 - [Medium Level Questions](#)
 - [Hard Level Questions](#)
 - [Bonus Patterns](#)
 - [Performance Considerations](#)
 - [Study Plan](#)
-

Easy Level Questions

1. Find Duplicate Records

Problem: Identify users with duplicate email addresses in a users table.

Data Engineering Context:

- Common data quality issue in ETL pipelines
- Critical for deduplication processes before loading into data warehouses
- Often used in data validation checks

```
sql

SELECT
  email,
  COUNT(*) as duplicate_count
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Why This Matters for DEs:

- **Data Quality:** Duplicates indicate upstream data issues
- **Pipeline Design:** Should be caught in data validation stage
- **Performance:** On large tables (millions of rows), consider:
 - Partitioning by date ranges
 - Using distributed computing (Spark, Presto)

- Creating hash indexes on email column

Production Considerations:

```
sql
-- Better approach for large datasets
SELECT
  email,
  COUNT(*) as duplicate_count,
  MIN(created_at) as first_occurrence,
  MAX(created_at) as last_occurrence
FROM users
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC
LIMIT 100; -- Limit results for performance
```

2. Second Highest Salary

Problem: Find the second highest salary from an employees table.

Data Engineering Context:

- Tests understanding of aggregation and subqueries
- Foundation for percentile calculations in analytics

```
sql
```

```

-- Method 1: Subquery approach
SELECT MAX(salary) as second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Method 2: LIMIT/OFFSET (better for large datasets)
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;

-- Method 3: Window function (most scalable)
WITH RankedSalaries AS (
    SELECT
        salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) as rank
    FROM employees
)
SELECT salary as second_highest
FROM RankedSalaries
WHERE rank = 2;

```

DE Performance Analysis:

- **Method 1:** Two full table scans - $O(2n)$
- **Method 2:** Single scan with sort - $O(n \log n)$ but efficient with index
- **Method 3:** Single scan, optimal for distributed systems

When to Use Each:

- Small tables (<10K rows): Any method works
- Large tables (millions): Method 3 with window functions
- Distributed systems (Spark/Hive): Window functions partition well

3. Delete Duplicates (Keep One)

Problem: Remove duplicate records while preserving one copy.

Data Engineering Context:

- Critical for maintaining data integrity in warehouses
- Common in CDC (Change Data Capture) pipelines
- Part of data deduplication workflows

sql

-- Standard approach

DELETE FROM employees

WHERE id NOT IN (

SELECT MIN(id)

FROM employees

GROUP BY email

);

-- Better approach for large tables (avoids subquery in DELETE)

WITH DuplicateRecords AS (

SELECT

id,

ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) as rn

FROM employees

)

DELETE FROM employees

WHERE id IN (

SELECT id FROM DuplicateRecords WHERE rn > 1

);

Production DE Pattern:

sql

-- Create cleaned table (avoids locks on production table)

CREATE TABLE employees_clean AS

SELECT DISTINCT ON (email) *

FROM employees

ORDER BY email, id;

-- Swap tables (minimal downtime)

BEGIN;

ALTER TABLE employees RENAME TO employees_backup;

ALTER TABLE employees_clean RENAME TO employees;

COMMIT;

Why This Approach:

- Avoids long-running DELETE locks
 - Maintains audit trail (backup table)
 - Faster on large datasets
 - Can be parallelized in distributed systems
-

4. Employees Earning More Than Their Managers

Problem: Find employees who earn more than their direct managers.

Data Engineering Context:

- Tests understanding of self-joins
- Common in hierarchical data analysis
- Foundation for graph queries

```
sql

SELECT
  e.name as Employee,
  e.salary as EmployeeSalary,
  m.name as Manager,
  m.salary as ManagerSalary
FROM employees e
INNER JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

DE Optimization for Large Datasets:

```
sql

-- Add indexes for performance
CREATE INDEX idx_employees_manager ON employees(manager_id);
CREATE INDEX idx_employees_salary ON employees(salary);

-- Query with additional context
SELECT
  e.department_id,
  COUNT(*) as employees_earning_more,
  AVG(e.salary - m.salary) as avg_difference
FROM employees e
INNER JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary
GROUP BY e.department_id;
```

Distributed Computing Consideration:

- Self-joins can be expensive in Spark/Hive
 - Consider broadcasting smaller dimension table (managers)
 - Use bucketing on manager_id for join optimization
-

Medium Level Questions

5. Nth Highest Salary (Generic Solution)

Problem: Find the Nth highest salary (e.g., 3rd, 5th, 10th highest).

Data Engineering Context:

- Foundation for percentile calculations
- Used in compensation analysis pipelines
- Tests window function knowledge

```
sql
-- Most efficient approach
WITH RankedSalaries AS (
  SELECT
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) as rank
  FROM employees
)
SELECT DISTINCT salary
FROM RankedSalaries
WHERE rank = 3; -- Change to any N
```

Why DENSE_RANK for DEs:

- `ROW_NUMBER()`: Every row gets unique rank (includes duplicates)
- `RANK()`: Skips ranks after ties (1,2,2,4,5)
- `DENSE_RANK()`: No gaps in ranking (1,2,2,3,4)  Best for salary questions

Parameterized Version (for data pipelines):

```
sql
```

```
-- PostgreSQL

CREATE OR REPLACE FUNCTION get_nth_highest_salary(n INTEGER)
RETURNS TABLE(salary NUMERIC) AS $$
BEGIN
    RETURN QUERY
    WITH RankedSalaries AS (
        SELECT
            e.salary,
            DENSE_RANK() OVER (ORDER BY e.salary DESC) as rank
        FROM employees e
    )
    SELECT r.salary
    FROM RankedSalaries r
    WHERE r.rank = n;
END;
$$ LANGUAGE plpgsql;

-- Usage
SELECT * FROM get_nth_highest_salary(3);
```

Performance Tips:

- Index on salary column: `CREATE INDEX idx_salary ON employees(salary DESC);`
- For very large tables, consider approximate percentiles: `PERCENTILE_CONT()`

6. Consecutive Numbers

Problem: Find numbers that appear at least 3 times consecutively.

Data Engineering Context:

- Pattern detection in time-series data
- Anomaly detection in logs
- Sequence analysis for fraud detection

```
sql
```

-- Classic self-join approach

```
SELECT DISTINCT l1.num as ConsecutiveNums
FROM Logs l1
INNER JOIN Logs l2 ON l1.id = l2.id - 1
INNER JOIN Logs l3 ON l1.id = l3.id - 2
WHERE l1.num = l2.num AND l2.num = l3.num;
```

-- Window function approach (better for DE)

```
WITH ConsecutiveGroups AS (
  SELECT
    num,
    id,
    id - ROW_NUMBER() OVER (PARTITION BY num ORDER BY id) as grp
  FROM Logs
)
SELECT DISTINCT num
FROM ConsecutiveGroups
GROUP BY num, grp
HAVING COUNT(*) >= 3;
```

Why Window Approach is Better:

- Single table scan instead of 3 self-joins
- Scales to any consecutive count (not just 3)
- More efficient on distributed systems

Real-World DE Use Case:

sql

-- Find users with 5+ consecutive login days

```
WITH LoginStreaks AS (  
  SELECT  
    user_id,  
    login_date,  
    login_date - ROW_NUMBER() OVER (  
      PARTITION BY user_id  
      ORDER BY login_date  
    ) * INTERVAL '1 day' as streak_group  
  FROM user_logins  
)  
SELECT  
  user_id,  
  MIN(login_date) as streak_start,  
  MAX(login_date) as streak_end,  
  COUNT(*) as consecutive_days  
FROM LoginStreaks  
GROUP BY user_id, streak_group  
HAVING COUNT(*) >= 5;
```

7. Department Top 3 Salaries

Problem: Find the top 3 highest-paid employees in each department.

Data Engineering Context:

- Common in analytics dashboards
- Tests PARTITION BY understanding
- Foundation for segmented analysis

sql

```
WITH RankedSalaries AS (  
  SELECT  
    d.name as Department,  
    e.name as Employee,  
    e.salary,  
    DENSE_RANK() OVER (  
      PARTITION BY e.department_id  
      ORDER BY e.salary DESC  
    ) as salary_rank  
  FROM employees e  
  INNER JOIN departments d ON e.department_id = d.id  
)  
SELECT  
  Department,  
  Employee,  
  Salary  
FROM RankedSalaries  
WHERE salary_rank <= 3  
ORDER BY Department, salary_rank;
```

DE Performance Optimization:

sql

-- Add indexes

```
CREATE INDEX idx_emp_dept_salary ON employees(department_id, salary DESC);
```

-- For very large datasets, consider filtering first

```
WITH FilteredEmployees AS (  
    SELECT *  
    FROM employees  
    WHERE hire_date >= '2020-01-01' -- Only recent employees  
)  
RankedSalaries AS (  
    SELECT  
        d.name as Department,  
        e.name as Employee,  
        e.salary,  
        DENSE_RANK() OVER (  
            PARTITION BY e.department_id  
            ORDER BY e.salary DESC  
        ) as salary_rank  
    FROM FilteredEmployees e  
    INNER JOIN departments d ON e.department_id = d.id  
)  
SELECT Department, Employee, Salary  
FROM RankedSalaries  
WHERE salary_rank <= 3;
```

Spark/Hive Optimization:

sql

-- Partition the result for better query performance

```
INSERT OVERWRITE TABLE top_salaries_by_dept
PARTITION(department_id)
SELECT
    employee_name,
    salary,
    salary_rank,
    department_id
FROM (
    SELECT
        name as employee_name,
        salary,
        department_id,
        DENSE_RANK() OVER (
            PARTITION BY department_id
            ORDER BY salary DESC
        ) as salary_rank
    FROM employees
) ranked
WHERE salary_rank <= 3;
```

8. Running Total / Cumulative Sum

Problem: Calculate running total of sales over time.

Data Engineering Context:

- Essential for financial reporting pipelines
- Common in metric dashboards
- Foundation for time-series analysis

sql

```
SELECT
    date,
    sales,
    SUM(sales) OVER (ORDER BY date) as running_total,
    SUM(sales) OVER (
        ORDER BY date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) as running_total_explicit
FROM daily_sales
ORDER BY date;
```

Advanced DE Pattern - Partitioned Running Total:

```
sql

-- Running total per store
SELECT
  store_id,
  date,
  sales,
  SUM(sales) OVER (
    PARTITION BY store_id
    ORDER BY date
  ) as store_running_total,
  SUM(sales) OVER (
    ORDER BY date
  ) as global_running_total
FROM daily_sales
ORDER BY store_id, date;
```

Performance Consideration:

```
sql

-- For very large time-series, use range instead of rows
SELECT
  date,
  sales,
  SUM(sales) OVER (
    ORDER BY date
    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) as running_total
FROM daily_sales;
```

Real-World DE Pipeline:

```
sql
```

-- Create materialized view for dashboard

CREATE MATERIALIZED VIEW mv_sales_running_totals AS

SELECT

date,

store_id,

region,

sales,

SUM(sales) OVER (

PARTITION BY store_id

ORDER BY date

) as store_cumulative,

SUM(sales) OVER (

PARTITION BY region

ORDER BY date

) as region_cumulative

FROM daily_sales;

-- Refresh nightly

REFRESH MATERIALIZED VIEW mv_sales_running_totals;

9. Month-over-Month Growth

Problem: Calculate month-over-month revenue growth.

Data Engineering Context:

- Key metric for business analytics
- Tests LAG/LEAD window functions
- Common in automated reporting pipelines

sql

SELECT

month,

revenue,

LAG(revenue) OVER (ORDER BY month) as prev_month_revenue,

revenue - LAG(revenue) OVER (ORDER BY month) as absolute_growth,

ROUND(

(revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0 /

NULLIF(LAG(revenue) OVER (ORDER BY month), 0),

2

) as growth_percentage

FROM monthly_revenue

ORDER BY month;

DE Best Practice - Handle Edge Cases:

sql

```
WITH MonthlyMetrics AS (  
  SELECT  
    DATE_TRUNC('month', order_date) as month,  
    SUM(amount) as revenue,  
    COUNT(DISTINCT customer_id) as customers  
  FROM orders  
  GROUP BY DATE_TRUNC('month', order_date)  
)  
SELECT  
  month,  
  revenue,  
  customers,  
  LAG(revenue) OVER (ORDER BY month) as prev_month_revenue,  
  COALESCE(  
    ROUND(  
      (revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0 /  
      NULLIF(LAG(revenue) OVER (ORDER BY month), 0),  
      2  
    ),  
    0  
  ) as growth_pct,  
  CASE  
    WHEN LAG(revenue) OVER (ORDER BY month) IS NULL THEN 'First Month'  
    WHEN revenue > LAG(revenue) OVER (ORDER BY month) THEN 'Growth'  
    WHEN revenue < LAG(revenue) OVER (ORDER BY month) THEN 'Decline'  
    ELSE 'Flat'  
  END as trend  
FROM MonthlyMetrics  
ORDER BY month;
```

Year-over-Year Comparison (Common in DE):

sql

```
SELECT
  month,
  revenue,
  LAG(revenue, 1) OVER (ORDER BY month) as mom_prev,
  LAG(revenue, 12) OVER (ORDER BY month) as yoy_prev,
  ROUND(
    (revenue - LAG(revenue, 1) OVER (ORDER BY month)) * 100.0 /
    NULLIF(LAG(revenue, 1) OVER (ORDER BY month), 0),
    2
  ) as mom_growth_pct,
  ROUND(
    (revenue - LAG(revenue, 12) OVER (ORDER BY month)) * 100.0 /
    NULLIF(LAG(revenue, 12) OVER (ORDER BY month), 0),
    2
  ) as yoy_growth_pct
FROM monthly_revenue
ORDER BY month;
```

10. Find Customers Who Never Ordered

Problem: Identify customers with no order history.

Data Engineering Context:

- Data quality check for customer engagement
- Common in churn analysis pipelines
- Tests understanding of LEFT JOIN vs NOT IN vs NOT EXISTS

sql


```
-- Method 1: LEFT JOIN (most readable)
SELECT c.customer_id, c.name, c.email
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- Method 2: NOT IN (avoid if orders.customer_id can be NULL)
SELECT customer_id, name, email
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders WHERE customer_id IS NOT NULL
);

-- Method 3: NOT EXISTS (best performance)
SELECT customer_id, name, email
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Performance Comparison (DE Perspective):

Method	Performance	Use Case
LEFT JOIN	Good	Best for readability, works well with indexes
NOT IN	Poor	Avoid if subquery can return NULL
NOT EXISTS	Best	Stops at first match, optimal for large tables

Production DE Query:

sql

-- Add business context for actionable insights

```
WITH InactiveCustomers AS (  
  SELECT  
    c.customer_id,  
    c.name,  
    c.email,  
    c.signup_date,  
    CURRENT_DATE - c.signup_date as days_inactive  
  FROM customers c  
  WHERE NOT EXISTS (  
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id  
  )  
)  
SELECT  
  customer_id,  
  name,  
  email,  
  signup_date,  
  days_inactive,  
  CASE  
    WHEN days_inactive <= 30 THEN 'Recently Signed Up'  
    WHEN days_inactive <= 90 THEN 'Needs Engagement'  
    ELSE 'At Risk Churn'  
  END as customer_segment  
FROM InactiveCustomers  
ORDER BY days_inactive DESC;
```

Hard Level Questions

11. Find Median Salary

Problem: Calculate the median (50th percentile) salary.

Data Engineering Context:

- Common in compensation analysis
- More robust than average for skewed distributions
- Tests understanding of percentiles

sql

-- Method 1: Built-in percentile function (PostgreSQL)

SELECT

PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) as median_salary

FROM employees;

-- Method 2: Manual calculation (works on all databases)

WITH OrderedSalaries AS (

SELECT

salary,

ROW_NUMBER() OVER (ORDER BY salary) as rn,

COUNT(*) OVER () as total_count

FROM employees

)

SELECT AVG(salary) as median_salary

FROM OrderedSalaries

WHERE rn IN (

FLOOR((total_count + 1) / 2.0),

CEIL((total_count + 1) / 2.0)

);

Why Manual Method Matters for DEs:

- Works on databases without PERCENTILE_CONT (MySQL, older SQLite)
- Understanding the logic helps with custom percentile calculations
- Can be adapted for weighted medians

Calculate Multiple Percentiles (Common in Analytics):

sql

SELECT

PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) as p25,

PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary) as median,

PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) as p75,

PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY salary) as p90,

PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY salary) as p95

FROM employees;

DE Pipeline Pattern:

sql

-- Store percentiles for daily monitoring

```
INSERT INTO salary_metrics (metric_date, p25, p50, p75, p90)
```

```
SELECT
```

```
    CURRENT_DATE,
```

```
    PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary),
```

```
    PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary),
```

```
    PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary),
```

```
    PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY salary)
```

```
FROM employees
```

```
WHERE is_active = true;
```

12. User Retention Rate

Problem: Calculate day-over-day user retention.

Data Engineering Context:

- Critical metric for product analytics
- Tests self-join with date logic
- Foundation for cohort analysis

sql

```
WITH UserActivity AS (
```

```
    SELECT DISTINCT
```

```
        user_id,
```

```
        DATE(activity_date) as activity_date
```

```
    FROM user_logs
```

```
)
```

```
SELECT
```

```
    a1.activity_date,
```

```
    COUNT(DISTINCT a1.user_id) as total_users,
```

```
    COUNT(DISTINCT a2.user_id) as retained_users,
```

```
    ROUND(
```

```
        COUNT(DISTINCT a2.user_id) * 100.0 /
```

```
        NULLIF(COUNT(DISTINCT a1.user_id), 0),
```

```
        2
```

```
    ) as retention_rate
```

```
FROM UserActivity a1
```

```
LEFT JOIN UserActivity a2
```

```
    ON a1.user_id = a2.user_id
```

```
    AND a2.activity_date = a1.activity_date + INTERVAL '1 day'
```

```
GROUP BY a1.activity_date
```

```
ORDER BY a1.activity_date;
```

Advanced: N-Day Retention (More Useful for PMs):

```
sql

-- 7-day retention
WITH UserActivity AS (
  SELECT DISTINCT
    user_id,
    DATE(activity_date) as activity_date
  FROM user_logs
)
SELECT
  a1.activity_date as cohort_date,
  COUNT(DISTINCT a1.user_id) as cohort_size,
  COUNT(DISTINCT CASE WHEN a2.activity_date = a1.activity_date + 1 THEN a2.user_id END) as day_1,
  COUNT(DISTINCT CASE WHEN a2.activity_date = a1.activity_date + 7 THEN a2.user_id END) as day_7,
  COUNT(DISTINCT CASE WHEN a2.activity_date = a1.activity_date + 30 THEN a2.user_id END) as day_30,
  ROUND(
    COUNT(DISTINCT CASE WHEN a2.activity_date = a1.activity_date + 7 THEN a2.user_id END) * 100.0 /
    NULLIF(COUNT(DISTINCT a1.user_id), 0),
    2
  ) as day_7_retention_rate
FROM UserActivity a1
LEFT JOIN UserActivity a2 ON a1.user_id = a2.user_id
GROUP BY a1.activity_date
ORDER BY a1.activity_date;
```

Production DE Pattern - Cohort Retention Table:

```
sql
```

-- Create a retention matrix

```
WITH FirstActivity AS (  
    SELECT  
        user_id,  
        MIN(DATE(activity_date)) as first_activity_date  
    FROM user_logs  
    GROUP BY user_id  
)  
ActivityByDay AS (  
    SELECT DISTINCT  
        user_id,  
        DATE(activity_date) as activity_date  
    FROM user_logs  
)  
SELECT  
    fa.first_activity_date as cohort,  
    COUNT(DISTINCT fa.user_id) as cohort_size,  
    COUNT(DISTINCT CASE  
        WHEN abd.activity_date BETWEEN fa.first_activity_date AND fa.first_activity_date + 7  
        THEN abd.user_id  
    END) as week_0_retained,  
    COUNT(DISTINCT CASE  
        WHEN abd.activity_date BETWEEN fa.first_activity_date + 7 AND fa.first_activity_date + 14  
        THEN abd.user_id  
    END) as week_1_retained,  
    COUNT(DISTINCT CASE  
        WHEN abd.activity_date BETWEEN fa.first_activity_date + 14 AND fa.first_activity_date + 21  
        THEN abd.user_id  
    END) as week_2_retained  
FROM FirstActivity fa  
LEFT JOIN ActivityByDay abd ON fa.user_id = abd.user_id  
GROUP BY fa.first_activity_date  
ORDER BY fa.first_activity_date;
```

13. Trip Cancellation Rate

Problem: Calculate trip cancellation rate excluding banned users.

Data Engineering Context:

- Real-world ride-sharing analytics
- Tests multiple JOIN conditions
- Common in operational dashboards

sql

```
SELECT
  request_date as Day,
  ROUND(
    SUM(CASE WHEN status LIKE 'cancelled%' THEN 1 ELSE 0 END) * 1.0 /
    COUNT(*),
    2
  ) as cancellation_rate
FROM trips t
INNER JOIN users c ON t.client_id = c.user_id AND c.banned = 'No'
INNER JOIN users d ON t.driver_id = d.user_id AND d.banned = 'No'
WHERE request_date BETWEEN '2023-10-01' AND '2023-10-03'
GROUP BY request_date
ORDER BY request_date;
```

DE Performance Optimization:

sql

```
-- Create filtered user views to avoid repeated JOINS
CREATE TEMPORARY TABLE active_users AS
SELECT user_id
FROM users
WHERE banned = 'No';

-- Optimized query
SELECT
  request_date as Day,
  COUNT(*) as total_trips,
  SUM(CASE WHEN status LIKE 'cancelled%' THEN 1 ELSE 0 END) as cancelled_trips,
  ROUND(
    SUM(CASE WHEN status LIKE 'cancelled%' THEN 1 ELSE 0 END) * 100.0 /
    COUNT(*),
    2
  ) as cancellation_rate_pct
FROM trips t
WHERE EXISTS (SELECT 1 FROM active_users WHERE user_id = t.client_id)
AND EXISTS (SELECT 1 FROM active_users WHERE user_id = t.driver_id)
AND request_date BETWEEN '2023-10-01' AND '2023-10-03'
GROUP BY request_date
ORDER BY request_date;
```

Real-World DE Extension:

sql

-- Add more business context

SELECT

```
request_date,
COUNT(*) as total_trips,
SUM(CASE WHEN status = 'completed' THEN 1 ELSE 0 END) as completed,
SUM(CASE WHEN status = 'cancelled_by_client' THEN 1 ELSE 0 END) as client_cancelled,
SUM(CASE WHEN status = 'cancelled_by_driver' THEN 1 ELSE 0 END) as driver_cancelled,
ROUND(AVG(trip_distance), 2) as avg_distance,
ROUND(AVG(trip_duration), 2) as avg_duration_mins,
ROUND(
    SUM(CASE WHEN status LIKE 'cancelled%' THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
    2
) as overall_cancellation_rate
FROM trips t
INNER JOIN users c ON t.client_id = c.user_id AND c.banned = 'No'
INNER JOIN users d ON t.driver_id = d.user_id AND d.banned = 'No'
WHERE request_date >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY request_date
ORDER BY request_date DESC;
```

14. Friends Recommendations (Graph Problem)

Problem: Find friends-of-friends who are not already direct friends.

Data Engineering Context:

- Graph traversal in SQL
- Social network analysis
- Foundation for recommendation systems

sql


```
-- Step 1: Normalize friendships (bidirectional)
WITH DirectFriends AS (
    SELECT user1_id as user_id, user2_id as friend_id FROM friendships
    UNION
    SELECT user2_id as user_id, user1_id as friend_id FROM friendships
),
-- Step 2: Find friends of friends
FriendsOfFriends AS (
    SELECT DISTINCT
        df1.user_id,
        df2.friend_id as potential_friend
    FROM DirectFriends df1
    INNER JOIN DirectFriends df2 ON df1.friend_id = df2.user_id
    WHERE df1.user_id != df2.friend_id
)
-- Step 3: Exclude existing friends
SELECT
    fof.user_id,
    fof.potential_friend as recommended_friend_id
FROM FriendsOfFriends fof
WHERE NOT EXISTS (
    SELECT 1
    FROM DirectFriends df
    WHERE df.user_id = fof.user_id
        AND df.friend_id = fof.potential_friend
);
```

Enhanced DE Version - Add Recommendation Score:

```
sql
```

```

WITH DirectFriends AS (
    SELECT user1_id as user_id, user2_id as friend_id FROM friendships
    UNION
    SELECT user2_id as user_id, user1_id as friend_id FROM friendships
),
FriendsOfFriends AS (
    SELECT
        df1.user_id,
        df2.friend_id as potential_friend,
        COUNT(*) as mutual_friends -- Number of mutual connections
    FROM DirectFriends df1
    INNER JOIN DirectFriends df2 ON df1.friend_id = df2.user_id
    WHERE df1.user_id != df2.friend_id
    GROUP BY df1.user_id, df2.friend_id
)
SELECT
    fof.user_id,
    fof.potential_friend,
    fof.mutual_friends,
    RANK() OVER (
        PARTITION BY fof.user_id
        ORDER BY fof.mutual_friends DESC
    ) as recommendation_rank
FROM FriendsOfFriends fof
WHERE NOT EXISTS (
    SELECT 1
    FROM DirectFriends df
    WHERE df.user_id = fof.user_id
        AND df.friend_id = fof.potential_friend
)
ORDER BY fof.user_id, recommendation_rank;

```

Performance at Scale (Millions of users):

sql

-- Create materialized view for faster queries

```
CREATE MATERIALIZED VIEW mv_friend_recommendations AS
WITH DirectFriends AS (
    SELECT user1_id as user_id, user2_id as friend_id FROM friendships
    UNION
    SELECT user2_id as user_id, user1_id as friend_id FROM friendships
),
FriendsOfFriends AS (
    SELECT
        df1.user_id,
        df2.friend_id as potential_friend,
        COUNT(*) as mutual_friends
    FROM DirectFriends df1
    INNER JOIN DirectFriends df2 ON df1.friend_id = df2.user_id
    WHERE df1.user_id != df2.friend_id
    GROUP BY df1.user_id, df2.friend_id
)
SELECT
    fof.user_id,
    fof.potential_friend,
    fof.mutual_friends
FROM FriendsOfFriends fof
WHERE NOT EXISTS (
    SELECT 1 FROM DirectFriends df
    WHERE df.user_id = fof.user_id AND df.friend_id = fof.potential_friend
)
AND fof.mutual_friends >= 2; -- Only recommend if 2+ mutual friends

-- Refresh daily
CREATE INDEX idx_recommendations ON mv_friend_recommendations(user_id, mutual_friends DESC);
```

15. Pivot Table - Sales by Product and Month

Problem: Convert row data to columnar format (pivot).

Data Engineering Context:

- Common for reporting and dashboards
- Tests CASE WHEN with aggregations
- Foundation for data transformation in ETL

sql

-- Manual pivot for specific months

SELECT

```
product_name,  
SUM(CASE WHEN MONTH(sale_date) = 1 THEN amount ELSE 0 END) as Jan,  
SUM(CASE WHEN MONTH(sale_date) = 2 THEN amount ELSE 0 END) as Feb,  
SUM(CASE WHEN MONTH(sale_date) = 3 THEN amount ELSE 0 END) as Mar,  
SUM(CASE WHEN MONTH(sale_date) = 4 THEN amount ELSE 0 END) as Apr,  
SUM(CASE WHEN MONTH(sale_date) = 5 THEN amount ELSE 0 END) as May,  
SUM(CASE WHEN MONTH(sale_date) = 6 THEN amount ELSE 0 END) as Jun
```

FROM sales

WHERE YEAR(sale_date) = 2024

GROUP BY product_name

ORDER BY product_name;

PostgreSQL Dynamic Pivot (Advanced DE):

sql

-- Using crosstab function

CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT *

FROM crosstab(

```
'SELECT product_name,  
      TO_CHAR(sale_date, "Mon") as month,  
      SUM(amount)  
FROM sales  
WHERE EXTRACT(YEAR FROM sale_date) = 2024  
GROUP BY product_name, TO_CHAR(sale_date, "Mon")  
ORDER BY product_name, MIN(sale_date)',  
'SELECT DISTINCT TO_CHAR(sale_date, "Mon")  
FROM sales  
WHERE EXTRACT(YEAR FROM sale_date) = 2024  
ORDER BY MIN(EXTRACT(MONTH FROM sale_date))'
```

) AS ct(

```
product_name TEXT,  
Jan NUMERIC, Feb NUMERIC, Mar NUMERIC, Apr NUMERIC,  
May NUMERIC, Jun NUMERIC, Jul NUMERIC, Aug NUMERIC,  
Sep NUMERIC, Oct NUMERIC, Nov NUMERIC, Dec NUMERIC
```

);

Production DE Pattern - Parameterized:

sql

-- Create reusable view

```
CREATE OR REPLACE VIEW v_monthly_sales_pivot AS
SELECT
    product_id,
    product_name,
    product_category,
    SUM(CASE WHEN month_num = 1 THEN amount END) as jan,
    SUM(CASE WHEN month_num = 2 THEN amount END) as feb,
    SUM(CASE WHEN month_num = 3 THEN amount END) as mar,
    SUM(CASE WHEN month_num = 4 THEN amount END) as apr,
    SUM(CASE WHEN month_num = 5 THEN amount END) as may,
    SUM(CASE WHEN month_num = 6 THEN amount END) as jun,
    SUM(CASE WHEN month_num = 7 THEN amount END) as jul,
    SUM(CASE WHEN month_num = 8 THEN amount END) as aug,
    SUM(CASE WHEN month_num = 9 THEN amount END) as sep,
    SUM(CASE WHEN month_num = 10 THEN amount END) as oct,
    SUM(CASE WHEN month_num = 11 THEN amount END) as nov,
    SUM(CASE WHEN month_num = 12 THEN amount END) as dec,
    SUM(amount) as total_annual
FROM (
    SELECT
        p.product_id,
        p.product_name,
        p.product_category,
        EXTRACT(MONTH FROM s.sale_date) as month_num,
        s.amount
    FROM sales s
    INNER JOIN products p ON s.product_id = p.product_id
    WHERE EXTRACT(YEAR FROM s.sale_date) = EXTRACT(YEAR FROM CURRENT_DATE)
) monthly_data
GROUP BY product_id, product_name, product_category;
```

16. Gap and Islands Problem

Problem: Find consecutive sequences in data.

Data Engineering Context:

- Critical for session analysis
- Identifies engagement streaks
- Used in time-series anomaly detection

-- Classic gap and islands approach

```
WITH DateRanges AS (  
  SELECT  
    user_id,  
    activity_date,  
    activity_date - ROW_NUMBER() OVER (  
      PARTITION BY user_id  
      ORDER BY activity_date  
    ) * INTERVAL '1 day' as island_id  
  FROM user_activity  
)  
SELECT  
  user_id,  
  MIN(activity_date) as streak_start,  
  MAX(activity_date) as streak_end,  
  COUNT(*) as consecutive_days  
FROM DateRanges  
GROUP BY user_id, island_id  
HAVING COUNT(*) >= 3 -- Find streaks of 3+ days  
ORDER BY user_id, streak_start;
```

How It Works (DE Interview Explanation):

1. `ROW_NUMBER()` assigns sequential numbers within each user
2. Subtracting `ROW_NUMBER` from date creates constant value for consecutive dates
3. Same `island_id` = consecutive sequence
4. `GROUP BY island_id` to find each sequence

Real-World Example - User Engagement Streaks:

sql

```

WITH DailyActivity AS (
  SELECT DISTINCT
    user_id,
    DATE(event_timestamp) as activity_date
  FROM events
  WHERE event_type IN ('login', 'purchase', 'interaction')
),
Streaks AS (
  SELECT
    user_id,
    activity_date,
    activity_date - ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY activity_date
    ) * INTERVAL '1 day' as streak_group
  FROM DailyActivity
),
StreakStats AS (
  SELECT
    user_id,
    MIN(activity_date) as streak_start,
    MAX(activity_date) as streak_end,
    COUNT(*) as days_consecutive,
    streak_group
  FROM Streaks
  GROUP BY user_id, streak_group
)
SELECT
  user_id,
  MAX(days_consecutive) as longest_streak,
  COUNT(*) as total_streaks,
  SUM(days_consecutive) as total_active_days
FROM StreakStats
WHERE days_consecutive >= 3
GROUP BY user_id
ORDER BY longest_streak DESC;

```

Production Pipeline:

sql

```

-- Create table for streak monitoring
CREATE TABLE user_engagement_streaks (
    user_id BIGINT,
    streak_start DATE,
    streak_end DATE,
    days_consecutive INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, streak_start)
);

-- Daily ETL to populate
INSERT INTO user_engagement_streaks (user_id, streak_start, streak_end, days_consecutive)
WITH DailyActivity AS (
    SELECT DISTINCT
        user_id,
        DATE(event_timestamp) as activity_date
    FROM events
    WHERE DATE(event_timestamp) = CURRENT_DATE - INTERVAL '1 day'
),
Streaks AS (
    SELECT
        user_id,
        activity_date,
        activity_date - ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY activity_date
        ) * INTERVAL '1 day' as streak_group
    FROM DailyActivity
)
SELECT
    user_id,
    MIN(activity_date),
    MAX(activity_date),
    COUNT(*)
FROM Streaks
GROUP BY user_id, streak_group
HAVING COUNT(*) >= 3
ON CONFLICT (user_id, streak_start) DO UPDATE
SET streak_end = EXCLUDED.streak_end,
    days_consecutive = EXCLUDED.days_consecutive;

```

17. Self Join - Manager Hierarchy

Problem: Display organizational hierarchy across multiple levels.

Data Engineering Context:

- Common in HR analytics
- Tests understanding of multiple self-joins
- Foundation for tree structure queries

sql

-- Show 3 levels of management

SELECT

e1.employee_id,
e1.name as Employee,
e2.name as Manager,
e3.name as Manager_of_Manager,
e4.name as Top_Manager

FROM employees e1

LEFT JOIN employees e2 ON e1.manager_id = e2.employee_id

LEFT JOIN employees e3 ON e2.manager_id = e3.employee_id

LEFT JOIN employees e4 ON e3.manager_id = e4.employee_id

ORDER BY e1.employee_id;

Better Approach - Recursive CTE (for any depth):

sql

```

WITH RECURSIVE EmployeeHierarchy AS (
  -- Anchor: Start with top-level employees (CEO, VPs)
  SELECT
    employee_id,
    name,
    manager_id,
    title,
    1 as level,
    CAST(name AS VARCHAR(1000)) as hierarchy_path
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive: Add subordinates level by level
  SELECT
    e.employee_id,
    e.name,
    e.manager_id,
    e.title,
    eh.level + 1,
    CAST(eh.hierarchy_path || '>' || e.name AS VARCHAR(1000))
  FROM employees e
  INNER JOIN EmployeeHierarchy eh ON e.manager_id = eh.employee_id
  WHERE eh.level < 10 -- Prevent infinite loops
)
SELECT
  employee_id,
  name,
  title,
  level as org_level,
  hierarchy_path
FROM EmployeeHierarchy
ORDER BY hierarchy_path;

```

Production DE Use Case - Org Chart Analytics:

sql

```

WITH RECURSIVE OrgTree AS (
  SELECT
    employee_id,
    name,
    manager_id,
    salary,
    department_id,
    1 as level,
    employee_id as root_manager_id
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  SELECT
    e.employee_id,
    e.name,
    e.manager_id,
    e.salary,
    e.department_id,
    ot.level + 1,
    ot.root_manager_id
  FROM employees e
  INNER JOIN OrgTree ot ON e.manager_id = ot.employee_id
)
SELECT
  root_manager_id,
  rm.name as executive,
  COUNT(*) as total_reports,
  AVG(salary) as avg_salary,
  MAX(level) as max_depth,
  SUM(salary) as total_payroll
FROM OrgTree ot
INNER JOIN employees rm ON ot.root_manager_id = rm.employee_id
GROUP BY root_manager_id, rm.name
ORDER BY total_reports DESC;

```

Span of Control Analysis (Manager effectiveness):

sql

```
WITH DirectReports AS (  
  SELECT  
    manager_id,  
    COUNT(*) as direct_reports,  
    AVG(salary) as avg_team_salary  
  FROM employees  
  WHERE manager_id IS NOT NULL  
  GROUP BY manager_id  
)  
SELECT  
  e.employee_id,  
  e.name as manager_name,  
  e.title,  
  dr.direct_reports,  
  dr.avg_team_salary,  
  CASE  
    WHEN dr.direct_reports <= 5 THEN 'Appropriate'  
    WHEN dr.direct_reports <= 10 THEN 'High'  
    ELSE 'Too Many'  
  END as span_assessment  
FROM employees e  
INNER JOIN DirectReports dr ON e.employee_id = dr.manager_id  
ORDER BY dr.direct_reports DESC;
```

18. Find Top Spending Customers by Category

Problem: Identify the highest-spending customer in each product category.

Data Engineering Context:

- Common in customer segmentation
- Tests PARTITION BY with RANK
- Used in personalization pipelines

sql

```
WITH CustomerSpending AS (  
  SELECT  
    c.customer_id,  
    c.customer_name,  
    p.category,  
    SUM(o.amount) as total_spent,  
    COUNT(o.order_id) as order_count,  
    RANK() OVER (  
      PARTITION BY p.category  
      ORDER BY SUM(o.amount) DESC  
    ) as spend_rank  
  FROM orders o  
  INNER JOIN customers c ON o.customer_id = c.customer_id  
  INNER JOIN products p ON o.product_id = p.product_id  
  GROUP BY c.customer_id, c.customer_name, p.category  
)  
SELECT  
  category,  
  customer_id,  
  customer_name,  
  total_spent,  
  order_count  
FROM CustomerSpending  
WHERE spend_rank = 1  
ORDER BY total_spent DESC;
```

Extended Analysis - Top 3 per Category:

sql

```

WITH CustomerCategorySpending AS (
  SELECT
    c.customer_id,
    c.customer_name,
    c.email,
    p.category,
    SUM(o.amount) as total_spent,
    COUNT(DISTINCT o.order_id) as order_count,
    AVG(o.amount) as avg_order_value,
    MIN(o.order_date) as first_purchase,
    MAX(o.order_date) as last_purchase,
    DENSE_RANK() OVER (
      PARTITION BY p.category
      ORDER BY SUM(o.amount) DESC
    ) as spend_rank
  FROM orders o
  INNER JOIN customers c ON o.customer_id = c.customer_id
  INNER JOIN products p ON o.product_id = p.product_id
  WHERE o.order_date >= CURRENT_DATE - INTERVAL '1 year'
  GROUP BY c.customer_id, c.customer_name, c.email, p.category
)
SELECT
  category,
  spend_rank,
  customer_name,
  email,
  total_spent,
  order_count,
  avg_order_value,
  CURRENT_DATE - last_purchase as days_since_purchase,
  CASE
    WHEN CURRENT_DATE - last_purchase <= 30 THEN 'Active'
    WHEN CURRENT_DATE - last_purchase <= 90 THEN 'At Risk'
    ELSE 'Churned'
  END as customer_status
FROM CustomerCategorySpending
WHERE spend_rank <= 3
ORDER BY category, spend_rank;

```

Production DE Pipeline - Customer Segmentation:

sql

-- Create materialized view for marketing campaigns

CREATE MATERIALIZED VIEW mv_customer_category_leaders AS

WITH CustomerMetrics AS (

SELECT

c.customer_id,

p.category,

SUM(o.amount) as lifetime_value,

COUNT(DISTINCT DATE_TRUNC('month', o.order_date)) as months_active,

MAX(o.order_date) as last_order_date

FROM orders o

INNER JOIN customers c ON o.customer_id = c.customer_id

INNER JOIN products p ON o.product_id = p.product_id

GROUP BY c.customer_id, p.category

),

RankedCustomers AS (

SELECT

*,

PERCENT_RANK() OVER (

PARTITION BY category

ORDER BY lifetime_value

) as percentile_rank

FROM CustomerMetrics

)

SELECT

customer_id,

category,

lifetime_value,

months_active,

last_order_date,

percentile_rank,

CASE

WHEN percentile_rank >= 0.95 THEN 'Platinum'

WHEN percentile_rank >= 0.80 THEN 'Gold'

WHEN percentile_rank >= 0.50 THEN 'Silver'

ELSE 'Bronze'

END as customer_tier

FROM RankedCustomers;

-- Refresh nightly

CREATE INDEX idx_customer_tier ON mv_customer_category_leaders(category, customer_tier);

19. Complex Date Logic - Active Subscriptions

Problem: Count active subscriptions for each calendar date.

Data Engineering Context:

- Critical for SaaS revenue recognition
- Tests date range overlap logic
- Common in financial reporting pipelines

sql

-- PostgreSQL version with generate_series

```
WITH DateSeries AS (  
    SELECT generate_series(  
        '2024-01-01'::date,  
        '2024-12-31'::date,  
        '1 day'::interval  
    )::date as calendar_date  
)  
SELECT  
    ds.calendar_date,  
    COUNT(DISTINCT s.user_id) as active_subscriptions,  
    SUM(s.monthly_price) as daily_mrr  
FROM DateSeries ds  
INNER JOIN subscriptions s  
    ON ds.calendar_date BETWEEN s.start_date AND COALESCE(s.end_date, '2099-12-31')  
GROUP BY ds.calendar_date  
ORDER BY ds.calendar_date;
```

More Efficient for Large Date Ranges:

sql

-- Only calculate for subscription change points

```
WITH SubscriptionEvents AS (  
    SELECT start_date as event_date, user_id, monthly_price, 1 as change_type  
    FROM subscriptions  
    UNION ALL  
    SELECT end_date + INTERVAL '1 day', user_id, monthly_price, -1 as change_type  
    FROM subscriptions  
    WHERE end_date IS NOT NULL  
)  
RunningTotal AS (  
    SELECT  
        event_date,  
        SUM(change_type) OVER (ORDER BY event_date) as active_count,  
        SUM(monthly_price * change_type) OVER (ORDER BY event_date) as mrr  
    FROM SubscriptionEvents  
)  
SELECT  
    event_date,  
    active_count as active_subscriptions,  
    mrr as monthly_recurring_revenue  
FROM RunningTotal  
ORDER BY event_date;
```

Production DE Pattern - Daily Snapshot:

sql

-- Create daily snapshot table

```
CREATE TABLE subscription_daily_metrics (  
  metric_date DATE PRIMARY KEY,  
  active_subscriptions INT,  
  new_subscriptions INT,  
  churned_subscriptions INT,  
  mrr DECIMAL(12,2),  
  arr DECIMAL(12,2),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Daily ETL job

```
INSERT INTO subscription_daily_metrics (  
  metric_date,  
  active_subscriptions,  
  new_subscriptions,  
  churned_subscriptions,  
  mrr,  
  arr  
)  
WITH ActiveSubs AS (  
  SELECT COUNT(DISTINCT user_id) as active_count  
  FROM subscriptions  
  WHERE CURRENT_DATE BETWEEN start_date AND COALESCE(end_date, '2099-12-31')  
)  
NewSubs AS (  
  SELECT COUNT(*) as new_count  
  FROM subscriptions  
  WHERE start_date = CURRENT_DATE  
)  
Churned AS (  
  SELECT COUNT(*) as churn_count  
  FROM subscriptions  
  WHERE end_date = CURRENT_DATE - INTERVAL '1 day'  
)  
Revenue AS (  
  SELECT SUM(monthly_price) as current_mrr  
  FROM subscriptions  
  WHERE CURRENT_DATE BETWEEN start_date AND COALESCE(end_date, '2099-12-31')  
)  
SELECT  
  CURRENT_DATE,  
  a.active_count,  
  n.new_count,  
  c.churn_count,  
  r.current_mrr,
```

```
r.current_mrr * 12
FROM ActiveSubs a, NewSubs n, Churned c, Revenue r
ON CONFLICT (metric_date) DO UPDATE
SET active_subscriptions = EXCLUDED.active_subscriptions,
    new_subscriptions = EXCLUDED.new_subscriptions,
    churned_subscriptions = EXCLUDED.churned_subscriptions,
    mrr = EXCLUDED.mrr,
    arr = EXCLUDED.arr;
```

20. Moving Average (Last 7 Days)

Problem: Calculate a 7-day rolling average of sales.

Data Engineering Context:

- Essential for smoothing time-series data
- Common in trend analysis dashboards
- Foundation for forecasting models

```
sql

SELECT
    date,
    sales,
    AVG(sales) OVER (
        ORDER BY date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) as moving_avg_7days,
    AVG(sales) OVER (
        ORDER BY date
        ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
    ) as moving_avg_30days
FROM daily_sales
ORDER BY date;
```

Advanced DE Pattern - Multiple Metrics:

```
sql
```

```

WITH DailySales AS (
  SELECT
    DATE(order_date) as date,
    SUM(amount) as daily_sales,
    COUNT(DISTINCT order_id) as order_count,
    COUNT(DISTINCT customer_id) as unique_customers
  FROM orders
  GROUP BY DATE(order_date)
)
SELECT
  date,
  daily_sales,
  order_count,
  unique_customers,
  -- 7-day moving averages
  ROUND(AVG(daily_sales) OVER w7, 2) as sales_ma7,
  ROUND(AVG(order_count) OVER w7, 2) as orders_ma7,
  ROUND(AVG(unique_customers) OVER w7, 2) as customers_ma7,
  -- 30-day moving averages
  ROUND(AVG(daily_sales) OVER w30, 2) as sales_ma30,
  -- Trend detection
  CASE
    WHEN AVG(daily_sales) OVER w7 > AVG(daily_sales) OVER w30 THEN 'Uptrend'
    WHEN AVG(daily_sales) OVER w7 < AVG(daily_sales) OVER w30 THEN 'Downtrend'
    ELSE 'Stable'
  END as trend
FROM DailySales
WINDOW
  w7 AS (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW),
  w30 AS (ORDER BY date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
ORDER BY date;

```

Weighted Moving Average (More sophisticated):

sql

-- Give more weight to recent data

```
WITH DailySales AS (  
  SELECT  
    DATE(order_date) as date,  
    SUM(amount) as daily_sales  
  FROM orders  
  GROUP BY DATE(order_date)  
)  
WeightedSales AS (  
  SELECT  
    date,  
    daily_sales,  
    ROW_NUMBER() OVER (ORDER BY date) as day_number  
  FROM DailySales  
)  
SELECT  
  date,  
  daily_sales,  
  -- Simple moving average  
  AVG(daily_sales) OVER (  
    ORDER BY date  
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
  ) as simple_ma7,  
  -- Exponential weighted (approximation)  
  SUM(daily_sales * POW(0.9, day_number - day_number)) OVER (  
    ORDER BY date  
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
  ) / SUM(POW(0.9, day_number - day_number)) OVER (  
    ORDER BY date  
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
  ) as weighted_ma7  
FROM WeightedSales  
ORDER BY date;
```

Production Pipeline - Anomaly Detection:

sql

-- Detect sales anomalies using moving average

```
WITH DailySales AS (  
  SELECT  
    DATE(order_date) as date,  
    SUM(amount) as daily_sales  
  FROM orders  
  GROUP BY DATE(order_date)  
,  
SalesWithStats AS (  
  SELECT  
    date,  
    daily_sales,  
    AVG(daily_sales) OVER (  
      ORDER BY date  
      ROWS BETWEEN 29 PRECEDING AND CURRENT ROW  
    ) as ma30,  
    STDDEV(daily_sales) OVER (  
      ORDER BY date  
      ROWS BETWEEN 29 PRECEDING AND CURRENT ROW  
    ) as stddev30  
  FROM DailySales  
)  
SELECT  
  date,  
  daily_sales,  
  ma30,  
  CASE  
    WHEN daily_sales > ma30 + (2 * stddev30) THEN 'Unusual High'  
    WHEN daily_sales < ma30 - (2 * stddev30) THEN 'Unusual Low'  
    ELSE 'Normal'  
  END as anomaly_status,  
  ROUND(((daily_sales - ma30) / NULLIF(ma30, 0)) * 100, 2) as pct_deviation  
FROM SalesWithStats  
WHERE date >= CURRENT_DATE - INTERVAL '90 days'  
ORDER BY date DESC;
```

Bonus Patterns

Pattern 1: String Manipulation

Extract domain from email addresses

```
sql
```

-- Basic extraction

```
SELECT
    email,
    SUBSTRING(email FROM POSITION('@' IN email) + 1) as domain
FROM users;
```

-- Advanced: Extract username and domain

```
SELECT
    email,
    SPLIT_PART(email, '@', 1) as username,
    SPLIT_PART(email, '@', 2) as domain,
    CASE
        WHEN email LIKE '%@gmail.com' THEN 'Consumer'
        WHEN email LIKE '%@%.edu' THEN 'Education'
        WHEN email NOT LIKE '%@gmail.com'
        AND email NOT LIKE '%@yahoo.com'
        AND email NOT LIKE '%@hotmail.com'
        THEN 'Business'
        ELSE 'Other Consumer'
    END as email_category
FROM users;
```

Clean and standardize phone numbers:

sql

-- Remove formatting, standardize to digits

```
SELECT
    phone_number,
    REGEXP_REPLACE(phone_number, '[^0-9]', '', 'g') as cleaned_phone,
    CASE
        WHEN LENGTH(REGEXP_REPLACE(phone_number, '[^0-9]', '', 'g')) = 10
        THEN '+1' || REGEXP_REPLACE(phone_number, '[^0-9]', '', 'g')
        ELSE REGEXP_REPLACE(phone_number, '[^0-9]', '', 'g')
    END as standardized_phone
FROM contacts;
```

Pattern 2: CASE WHEN for Categorization

Customer segmentation by spending

sql

```

WITH CustomerSpending AS (
    SELECT
        customer_id,
        SUM(amount) as total_spent,
        COUNT(DISTINCT order_id) as order_count,
        MAX(order_date) as last_order_date
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    total_spent,
    order_count,
    CASE
        WHEN total_spent >= 10000 THEN 'VIP'
        WHEN total_spent >= 5000 THEN 'Premium'
        WHEN total_spent >= 1000 THEN 'Standard'
        ELSE 'Basic'
    END as customer_tier,
    CASE
        WHEN CURRENT_DATE - last_order_date <= 30 THEN 'Active'
        WHEN CURRENT_DATE - last_order_date <= 90 THEN 'At Risk'
        ELSE 'Inactive'
    END as activity_status,
    CASE
        WHEN order_count >= 20 THEN 'Loyal'
        WHEN order_count >= 10 THEN 'Regular'
        WHEN order_count >= 3 THEN 'Repeat'
        ELSE 'New'
    END as loyalty_segment
FROM CustomerSpending;

```

RFM Segmentation (Recency, Frequency, Monetary):

sql


```

WITH CustomerRFM AS (
  SELECT
    customer_id,
    CURRENT_DATE - MAX(order_date) as recency,
    COUNT(DISTINCT order_id) as frequency,
    SUM(amount) as monetary
  FROM orders
  GROUP BY customer_id
),
RFMScores AS (
  SELECT
    customer_id,
    recency,
    frequency,
    monetary,
    NTILE(5) OVER (ORDER BY recency DESC) as r_score,
    NTILE(5) OVER (ORDER BY frequency) as f_score,
    NTILE(5) OVER (ORDER BY monetary) as m_score
  FROM CustomerRFM
)
SELECT
  customer_id,
  r_score,
  f_score,
  m_score,
  r_score + f_score + m_score as rfm_score,
  CASE
    WHEN r_score >= 4 AND f_score >= 4 AND m_score >= 4 THEN 'Champions'
    WHEN r_score >= 3 AND f_score >= 3 THEN 'Loyal Customers'
    WHEN r_score >= 4 AND f_score <= 2 THEN 'New Customers'
    WHEN r_score <= 2 AND f_score >= 3 THEN 'At Risk'
    WHEN r_score <= 2 AND f_score <= 2 THEN 'Lost'
    ELSE 'Potential'
  END as customer_segment
FROM RFMScores;

```

Pattern 3: Recursive CTE (Employee Hierarchy)

Full organizational tree with all levels

```
sql
```

```

WITH RECURSIVE EmployeeHierarchy AS (
  -- Anchor: Start with CEO (no manager)
  SELECT
    employee_id,
    name,
    manager_id,
    title,
    salary,
    1 as level,
    CAST(name AS VARCHAR(500)) as org_path,
    ARRAY[employee_id] as path_array
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive: Add direct reports
  SELECT
    e.employee_id,
    e.name,
    e.manager_id,
    e.title,
    e.salary,
    eh.level + 1,
    CAST(eh.org_path || '>' || e.name AS VARCHAR(500)),
    eh.path_array || e.employee_id
  FROM employees e
  INNER JOIN EmployeeHierarchy eh
    ON e.manager_id = eh.employee_id
  WHERE NOT e.employee_id = ANY(eh.path_array) -- Prevent cycles
    AND eh.level < 20 -- Safety limit
)
SELECT
  employee_id,
  REPEAT(' ', level - 1) || name as indented_name,
  title,
  level as org_level,
  org_path,
  salary
FROM EmployeeHierarchy
ORDER BY org_path;

```

Calculate total team size and payroll:

```
sql
```

```
WITH RECURSIVE TeamHierarchy AS (  
  SELECT  
    employee_id,  
    manager_id,  
    salary,  
    1 as team_size,  
    salary as team_payroll  
  FROM employees  
  
  UNION ALL  
  
  SELECT  
    e.employee_id,  
    e.manager_id,  
    e.salary,  
    1 + COALESCE(SUM(th.team_size), 0),  
    e.salary + COALESCE(SUM(th.team_payroll), 0)  
  FROM employees e  
  LEFT JOIN TeamHierarchy th ON e.employee_id = th.manager_id  
  GROUP BY e.employee_id, e.manager_id, e.salary  
)  
SELECT  
  e.employee_id,  
  e.name,  
  e.title,  
  th.team_size as total_reports,  
  th.team_payroll as total_team_cost  
FROM TeamHierarchy th  
INNER JOIN employees e ON th.employee_id = e.employee_id  
WHERE th.team_size > 1  
ORDER BY th.team_size DESC;
```

Performance Considerations

Indexing Strategy

sql

```

-- Single column indexes
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_orders_amount ON orders(amount);

-- Composite indexes (column order matters!)
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);
CREATE INDEX idx_orders_date_status ON orders(order_date, status);

-- Covering index (includes all needed columns)
CREATE INDEX idx_orders_covering ON orders(customer_id, order_date)
INCLUDE (amount, status);

-- Partial index (only index subset of data)
CREATE INDEX idx_orders_recent ON orders(order_date)
WHERE order_date >= CURRENT_DATE - INTERVAL '1 year';

-- Expression index
CREATE INDEX idx_users_email_domain ON users(
  LOWER(SPLIT_PART(email, '@', 2))
);

```

Query Optimization Tips

1. Use EXISTS instead of IN for large subqueries

```

sql

-- Slow
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

-- Fast
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

```

2. Avoid SELECT * in production

```

sql

-- Slow (retrieves unnecessary data)
SELECT * FROM orders WHERE order_date = CURRENT_DATE;

-- Fast (only needed columns)
SELECT order_id, customer_id, amount FROM orders
WHERE order_date = CURRENT_DATE;

```

3. Use LIMIT for large result sets

```
sql

-- Add LIMIT to paginate
SELECT * FROM orders
ORDER BY order_date DESC
LIMIT 100 OFFSET 0;
```

4. Partition large tables

```
sql

-- PostgreSQL table partitioning by date
CREATE TABLE orders (
  order_id BIGINT,
  order_date DATE,
  amount DECIMAL(10,2)
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2024_q1 PARTITION OF orders
FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');

CREATE TABLE orders_2024_q2 PARTITION OF orders
FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');
```

Study Plan

Week 1: Foundations (Easy + Medium)

- Day 1-2: Aggregations, GROUP BY, HAVING (Q1-Q4)
- Day 3-4: Window Functions basics (Q5, Q7, Q8)
- Day 5: JOINS and subqueries (Q10)
- Day 6-7: Practice and review

Week 2: Advanced Concepts (Medium + Hard)

- Day 1-2: LAG/LEAD, date logic (Q9, Q12)
- Day 3-4: Complex window functions (Q6, Q16)
- Day 5: Percentiles and statistics (Q11, Q20)
- Day 6-7: Practice medium questions

Week 3: Expert Level (Hard)

- Day 1-2: Recursive CTEs (Q17, Pattern 3)
- Day 3: Graph problems (Q14)
- Day 4: Complex business logic (Q13, Q15, Q19)
- Day 5-7: Mock interviews, review weak areas

Week 4: Production Readiness

- Performance optimization
 - Real-world DE patterns
 - Mock interview practice
 - Review all questions
-

Must-Know Concepts

✅ Window Functions

- `ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`
- `LAG()`, `LEAD()`
- `PARTITION BY` vs `GROUP BY`
- Frame clauses: `ROWS BETWEEN`, `RANGE BETWEEN`

✅ CTEs (Common Table Expressions)

- Readability and reusability
- Recursive CTEs for hierarchies
- Multiple CTEs in one query

✅ JOINS

- INNER vs LEFT vs RIGHT vs FULL
- Self-joins for hierarchical data
- CROSS JOIN for combinations
- Performance: JOIN vs subquery

✅ Aggregations

- `GROUP BY` with multiple columns
- `HAVING` for filtered aggregations
- `DISTINCT` vs `GROUP BY`

✓ Date/Time Functions

- `DATE_TRUNC()`, `EXTRACT()`
- `INTERVAL` arithmetic
- Timezone handling

✓ String Functions

- `SUBSTRING()`, `SPLIT_PART()`
- `REGEXP_REPLACE()`, `REGEXP_MATCH()`
- `CONCAT()`, `LOWER()`, `UPPER()`

✓ Performance

- Index strategy
 - Query execution plans (`EXPLAIN()`)
 - Partitioning strategies
 - Materialized views
-

Interview Tips

1. Always clarify requirements

- "Should we handle NULL values?"
- "Do you want ties included?"
- "What's the expected data volume?"

2. Explain your approach

- "I'm using `ROW_NUMBER` because..."
- "I chose `LEFT JOIN` over `NOT IN` because..."
- "For large datasets, I'd add indexes on..."

3. Think like a DE

- Consider data volume and scalability
- Mention partitioning for large tables
- Discuss materialized views for frequent queries
- Talk about incremental updates vs full refreshes

4. Code quality

- Use meaningful CTEs names

- Add comments for complex logic
- Format for readability
- Handle edge cases (NULLs, empty results)

5. Production considerations

- How would this run in a daily ETL?
 - What if the table has 100M rows?
 - How to monitor this query?
 - What indexes would help?
-

Additional Resources

- **Practice Platforms:** LeetCode, HackerRank, SQLZoo
 - **Documentation:** PostgreSQL, MySQL, SQL Server official docs
 - **Books:** "SQL Performance Explained" by Markus Winand
 - **Tools:** pgAdmin, DBeaver, DataGrip for local practice
-

Good luck with your Data Engineering interviews! 🚀

Remember: It's not just about knowing SQL syntax, but understanding when and how to apply these patterns efficiently in production systems.